

로보틱스 구현을 위한

# 센서/액추에이터 시뮬레이션 프로그래밍 기본

이 속 헌

(주)러닝스페이스

[tonaidolee@naver.com](mailto:tonaidolee@naver.com)

## 2일차 다룰 내용

### ◆ 다관절 로봇을 위한 이론

- 다변수 함수, 다변수 함수와 벡터표현, 내적, 외적
- 편미분과 기하학적 의미, 연쇄법칙
- 열 벡터, 다변수 벡터 함수의 미분

### ◆ 개발 환경 구성

- 로봇 개발에 활용되는 개발 환경들
- 사용할 환경 구성

### ◆ ROS2

- 구조도
- 필수 개념

# 로봇 구현을 위한 수학



# 다변수 함수

## ◆ 로봇의 위치 제어: 다변수 함수, 다변수 벡터 함수

### ➤ 다변수 함수

#### ◆ 다변수 함수 : $y=f(x_1, x_2, \dots, x_n)$

- ▶ 변수가 여러 개인 함수 (여러 개의 변수에 의존하는 함수)

Ex)  $f(x,y)=x^2+2xy$

- ▶ 다변수 함수처럼  $n$ 개의 변수에서 하나의 값, 하나의 함수로 갈 때, ' $R^n \rightarrow R$ '로 표현

Ex)  $f(x,y)=x^2+2xy$ 와 같은 경우,  $x$ 하고  $y$ 가 두 개의 변수 존재.

$R^2$ 가 원래  $R$ 로 갔다는 의미 :  $R^2 \rightarrow R$  로 표현

### ➤ 다변수 벡터 함수

#### ◆ 다변수 벡터 함수

- ▶  $R^n \rightarrow R^m$ 으로 가는 함수도 있겠죠. 함수가 하나가 있는 게 아니라 여러 개 필요
- ▶  $f_1(x_1, x_2, \dots, x_n), f_2(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n)$ : 다변수 함수가 여러 개인 경우  
-> '다변수 벡터 함수'

Ex)  $(x_1^2, x_1^2+2x_1x_2, x_2x_3^2)$ 의 경우, 함수가 셋이 있는데 함수도 여러 개

변수도  $x_1, x_2, x_3$  총 3개가 있으므로 다변수 벡터 함수

# 다변수 함수의 편미분

## ◆ 로봇의 위치 제어: 다변수 함수 편미분

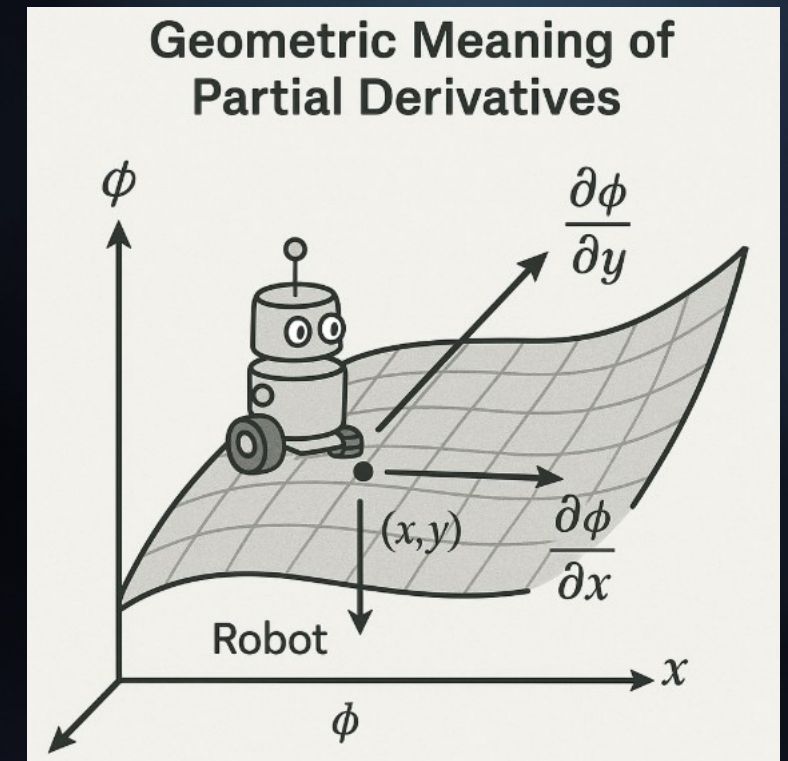
### ➤ 편미분

- ✓ 다변수 함수에서 미분하는 방법
- ✓ 다변수함수의 여러 변수 중, 하나의 변수에 대해서 관심을 갖고 미분하는 것
- ✓ Ex)  $x, y$ 의 2가지 변수에 의존하는 함수인 이변수 함수,  $z=f(x, y)$ 를 미분할 때 한 변수만 골라서 미분
- ✓  $\frac{\partial f}{\partial x}, \frac{\partial z}{\partial x}, f_x, D_x f$
- ✓ 편미분 기호 :  $\partial$ 를 사용 (라운드, 파셜)
- ✓ 편미분 비교로 이전 미분을 전미분

# 다변수 함수의 편미분

## ◆ 편미분의 기하학적 의미

- 전미분에서 기울기 의미처럼 편미분도 기울기라는 의미
- 미분과 기울기인데, 하나의 편도  $x$ 냐  $y$ 냐 이렇게 한 가지만 봐서, 다른 평면에 따라서 나올 곡선의 기울기를 본 것
  - 잘라서 기울기를 나타낸다는 의미
- 특정 점에서
  - ✓  $x$ 에 대한 편미분 →  $y$ 를 고정하고,  $x$  방향으로 자른 단면의 기울기
  - ✓  $y$ 에 대한 편미분 →  $x$ 를 고정하고,  $y$  방향으로 자른 단면의 기울기
  - 즉, 편미분은 곡면을 하나의 평면으로 잘라 얻은 곡선의 기울기를 나타냅니다.





# 다변수 함수의 편미분

## ◆ 연쇄 법칙

- 변수가 또 다른 변수에 연속적으로 의존할 경우, 이것 미분하는 방법
- 일변수 함수에서 미분할 때 체인을 많이 사용

ex)  $y=h(x)$ 의 변수,  $x=f(t)$ 일 때  $y$ 를  $t$ 에 대해서 미분한다

→ 실제로  $y$ 가 이렇게  $x$ 에 대해 있으면  $t$ 가 없으므로  $t$ 로 바로 미분할 수 없음.

→ 이  $h$ 가  $x$ 에 대한 함수니까  $x$ 에 대해서 미분

→  $t$ 에 대한 미분이 필요 →  $x$ 를 다시 한 번  $t$ 에 대해서 미분해 주는 이런 과정필요

: 이런 식으로 체인, 연쇄적으로 미분하는 걸 chain rule이라고 함.

# 개발 환경 구성





# 로봇 개발에 사용되는 개발 환경들

## ◆ 시스템 구조 비교

| 구분       | Ubuntu + ROS2 + Gazebo                                   | 물리엔진/시뮬레이터 + 동역학/제어 라이브러리 + 메시징/HIL 구성                                     |
|----------|----------------------------------------------------------|----------------------------------------------------------------------------|
| 구성요소     | ROS2 미들웨어 + Gazebo 물리엔진 + URDF/xacro 모델 + rviz, rqt 등 도구 | 물리엔진(MuJoCo/Drake 등) + 동역학/제어 라이브러리(Pinocchio, RBDL 등) + 통신 (LCM/ZeroMQ 등) |
| 통신 방식    | DDS 기반 Publish/Subscribe 메시징 (토픽, 서비스, 액션)               | 소켓/LCM/ZeroMQ 기반 직접 통신, 또는 함수 호출 기반                                        |
| 모듈화 수준   | 매우 높음 (센서, 제어기, 네비게이션 등 ROS 패키지 단위)                      | 사용자가 직접 구조 설계 (자유도가 높음)                                                    |
| 개발 언어    | Python/C++ (ROS 인터페이스 필수)                                | C++ / Python / Matlab 등 자유                                                 |
| 시뮬레이션 엔진 | Gazebo (ODE, DART, Bullet 선택 가능)                         | MuJoCo, Drake, Webots, CoppeliaSim 등 다양                                    |
| 하드웨어 연동  | ROS 드라이버 패키지(EtherCAT, CANopen, Serial 등)                | SOEM 등 네이티브 마스터 직접 제어 가능                                                   |
| 운영체제 호환성 | Linux 중심 (Ubuntu 권장)                                     | Windows / Linux / macOS 모두 지원 가능                                           |
| 실시간성     | DDS 기반이라 제어 루프는 실시간 보장 어려움                               | 제어기 직접 작성 시 실시간 루프(1~2kHz) 구현 가능                                           |
| 확장성      | 센서·로봇 추가가 쉬움 (패키지 재활용)                                   | 직접 모델 구성 필요 (코드 수준에서 관리)                                                   |

# 로봇 개발에 사용되는 개발 환경들

## ◆ 개발자 입장에서의 구조적 차이

| 구분        | ROS2 기반 구성                                    | ROS2 없이 구성                              |
|-----------|-----------------------------------------------|-----------------------------------------|
| 아키텍처      | DDS 기반의 메시지 중심 미들웨어 (Publisher-Subscriber 구조) | 직접 제어 루프·통신·동역학 코드를 작성 (직접 소켓/라이브러리 호출) |
| 개발 환경 복잡도 | 패키지, 토픽, 서비스, 액션, launch 파일 관리 필요             | 단일 프로세스 내 코드 제어, 디버깅 단순                 |
| 언어        | Python/C++ (rclpy, rclcpp)                    | C++/Python/Matlab 등 자유                  |
| 통신 구조     | 비동기 DDS, 네트워크 분산 가능 (분산 제어에 강점)               | 단일 머신 실시간 제어 유리                         |
| 제어 루프     | ROS callback 기반 (실시간성 낮음)                     | 실시간 제어(1~2kHz) 가능 (Thread, RT 커널 사용)    |
| 시각화/디버깅   | rviz, rqt, rosbag 등 풍부한 툴                     | 직접 로그/그래프 구현 필요                         |
| 시뮬레이터 통합  | Gazebo / Isaac Sim / Webots 등 ROS 플러그인 사용     | MuJoCo/Drake/Webots API 직접 호출           |
| 하드웨어 연동   | ROS 드라이버(EtherCAT, CAN, Serial 등) 이용          | SOEM 등 네이티브 드라이버 직접 구현                  |
| 코드 이식성    | ROS2가 설치된 환경 필요                               | OS 종속성 낮음 (Windows·Linux 공용)            |
| 학습 대상     | 토픽, 메시지 타입, QoS 등 개념 숙지 필요                    | 제어/동역학 수학적 지식                           |

# 로봇 개발에 사용되는 개발 환경들

## ◆ 장단점

| 구분      | ROS2 기반 구성                  |                           | ROS2 없이 구성              |                              |
|---------|-----------------------------|---------------------------|-------------------------|------------------------------|
|         | 장점                          | 단점                        | 장점                      | 단점                           |
| 생산성     | 수많은 오픈소스 패키지 활용, 빠른 프로토타이핑  | 패키지 버전·빌드 충돌, 복잡한 환경설정    | 코드 간결, 디버깅 직관적          | 모든 기능(센서, 제어, UI)을 직접 구현해야 함 |
| 통신      | 분산 시스템/다중 로봇 연결 용이          | DDS 지연·비결정성 → 실시간 제어엔 부적합 | 단일 머신 실시간 루프 안정적        | 네트워크 확장 시 직접 프로토콜 설계 필요      |
| 시각화     | rviz, Gazebo, rqt 등 강력한 시각화 | GUI 종속적, 커스터마이징 어려움       | MuJoCo/Drake 시각화 빠르고 경량 | 고급 GUI 기능은 직접 구현 필요          |
| 실시간 제어  | 실시간 제어 불가(DDS 구조상 한계)       |                           | 실시간 제어(1~2kHz) 가능       | 스케줄러·RTOS 설정 필요              |
| 유지 보수   | 커뮤니티 업데이트, 표준화된 메시지 구조      | ROS 버전 변경 시 API 파괴적 변화 잦음 | 단순한 코드베이스, 버전 영향 적음     | 유지보수·확장성은 개발자 의존적            |
| 학습 및 협업 | 로봇 연구자/교육에 표준화되어 있음         | 고성능 상용 로봇 제어엔 부적합         | 임베디드·산업 제어 팀 선호         | 신규 인력 온보딩 어려움                |

# 로봇 개발에 사용되는 개발 환경들

## ◆ 실제 활용 패턴 비교 (산업/연구 기준)

| 구분                     | Ubuntu + ROS2 + Gazebo                                             | 물리엔진/시뮬레이터 + 동역학/제어 라이브러리 + 메시징/HIL 구성                                 |
|------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------|
| 핵심 개념                  | 통합 생태계                                                             | 최고 성능 부품의 조합                                                           |
| 교육/연구용                 | ROS 표준 예제, SLAM, MoveIt2, Navigation 등 활용 쉬움                       | 로봇 제어 알고리즘, 강화학습, 제어이론 검증에 적합                                          |
| 산업용 개발<br>(양산 전 시뮬레이션) | ROS2는 테스트 환경으로 제한적 사용                                              | 실제 제어기 펌웨어/HIL 검증용 표준 구조                                               |
| 임베디드/펌웨어 팀             | ROS 의존도 낮고 DDS 부담 때문에 선호 안함                                        | 펌웨어 수준에서 EtherCAT 직접 제어, 실시간성 확보                                       |
| AI/ML 기반 강화학습          | Isaac Sim (ROS2 통합) 등 GPU 가속형 사용 가능                                | MuJoCo, PyBullet 등 경량 엔진으로 빠른 학습 실험                                    |
| 서비스/데모 환경              | ROS2 + Rviz로 시각화/서비스 배포 쉬움                                         | GUI 직접 구현 필요                                                           |
| 산업 표준화/생산라인            | ROS는 보안·실시간성·표준 미흡으로 비권장                                           | RobotStudio, ROBOGUIDE 등 독립 시뮬레이터 주류                                   |
| 주 사용처                  | 산업현장, 사용 로봇 개발, 물류 로봇, 서비스 로봇 개발 등                                 | AI연구소, 대학, 고난도 제어 알고리즘 개발 등                                            |
| 장점                     | 뛰어난 확장성 및 통합성, 방대한 커뮤니티와 자료<br>시뮬레이션 → 실제 로봇 전환 용이, 다양한 센서/하드웨어 지원 | 매우 빠르고 정확한 물리 연산, 정밀한 접촉(Contact) 동역학, 강화학습, 제어 연구에 최적화                |
| 단점                     | 상대적으로 무겁고 느린 물리 연산<br>복잡한 접촉 시뮬레이션의 정확도 한계                         | ROS만큼 통합적인 생태계 부재, 하드웨어 연동, 센서 통합 등 직접 구현 필요<br>학습 자료 및 커뮤니티가 상대적으로 작음 |
| 한 줄 표현                 | "로봇 개발의 A to Z를 책임지는 만능 프레임워크"                                     | "로봇의 물리와 제어 '코어'에 집중하는 전문가용 툴킷"                                        |

# 로봇 개발에 사용되는 개발 환경들

## ◆ 실제 기업/연구소 사례

| 분야                                 | 활용 도구                               | 특징                           |
|------------------------------------|-------------------------------------|------------------------------|
| 자동차 OEM/ 협동로봇 기업                   | CoppeliaSim + 자체 제어기 + EtherCAT     | ROS 없이 로우 레벨 제어 및 실시간 시뮬레이션  |
| Boston Dynamics / Agility Robotics | MuJoCo, Drake, Simscape Multibody   | 물리엔진 직접 사용, ROS는 서비스 포맷으로만   |
| ETH Zurich, MIT CSAIL              | Drake + Pinocchio                   | 제어 알고리즘, Jacobian 기반 역기구학 검증 |
| NVIDIA / Open Robotics             | Isaac Sim + ROS2                    | 연구/시각화 목적, AI-기반 로봇학습 환경     |
| FANUC / ABB / KUKA                 | 자사 전용 시뮬레이터(RobotStudio, ROBOGUIDE) | 상용 로봇용 네이티브 시뮬레이터, ROS 미사용   |

# 로봇 개발에 사용되는 개발 환경들

## ◆ 목적에 따른 환경 선택

| 목적                    | 환경 구성 예                           |
|-----------------------|-----------------------------------|
| 로봇팔 기본 학습 / ROS 네트워킹  | Ubuntu + ROS2 + Gazebo            |
| 제어 알고리즘 개발 / 실시간 테스트  | MuJoCo / Drake + Pinocchio / RBDL |
| 산업용 제어기 / EtherCAT 통합 | SOEM + CoppeliaSim / FMI 기반 HIL   |
| 강화학습 / 시뮬레이션 가속       | MuJoCo / PyBullet (GPU 지원)        |
| 제품화 / 생산라인 검증         | RobotStudio / ROBOGUIDE 등 상용툴     |

→ "ROS2 + Gazebo" : 상위 계층(통합, 시각화, 네트워크 중심) 개발용

→ "네이티브 시뮬레이션" : 하위계층(제어기, 동역학, 펌웨어 검증) 개발용

→ "네이티브 + ROS2래퍼" 형태로 분리되어 사용 : 하위 제어기 검증 = 네이티브, 서비스 제공 = ROS2 래퍼



# 로봇 개발에 사용되는 개발 환경들

## ◆ 실무용 로봇틱스 개발 시뮬레이션 환경 구성

| 구분         | 선택 기술 / 툴                                 | 역할                                   | 비고             |
|------------|-------------------------------------------|--------------------------------------|----------------|
| 언어 / IDE   | Python 3.10+, Visual Studio Code, Jupyter | 알고리즘/제어 로직 작성 및 시각화                  | cross-platform |
| 기구학/동역학 엔진 | Pinocchio + RBDL                          | FK/IK, Jacobian, RNEA, ABA 등 구현 및 검증 | 고성능, ROS 독립    |
| 물리 시뮬레이터   | MuJoCo 3.0                                | 로봇 물리 환경, 관절 시뮬, 센서 모델링, 시각화         | 실시간성 우수        |
| 비전/센서 처리   | OpenCV, NumPy, Matplotlib                 | 카메라, IMU, 거리센서 데이터 처리                | 실제-시뮬 양쪽 적용    |
| 제어 실습용 환경  | Control Toolbox (Python-Control)          | PID, 상태공간, 주파수응답, 제어 설계 실습           | SciPy 기반       |
| 머신러닝       | Scikit-learn, TensorFlow-lite             | 로봇 동작 패턴 학습, 비전 기반 인식                | 실무형 알고리즘 실습    |
| 데이터 통신/로깅  | ZeroMQ / LCM                              | 센서-제어기 간 데이터 송수신 구조 학습               | ROS 없이 IPC 학습  |
| 시각화 / UI   | Matplotlib 3D, Plotly Dash (선택)           | 궤적, 제어 응답, 3D Pose 시각화               | 대화형 실습 가능      |

# 개발 환경 구성

## Gazebo 시뮬레이션

(물리 환경 및 시각화 계층  
- 로봇과 환경 상호작용)

**역할:** 물리 엔진과 시각화를 담당

로봇 모델(URDF/SDF)을 실제 환경에 배치하고, 중력, 마찰, 관성 등의 물리 법칙을 시뮬레이션 수행  
ROS2와의 통신을 통해 센서 데이터(Publisher)와 제어 명령(Subscriber)를 주고 받음  
실제 로봇 제어 시스템을 구현하기 전, 가상 환경에서 안전하게 테스트할 수 있음

## Movel2

(모션 플래닝, 경로 생성,  
토픽/액션 기반으로 명령 송신)

**역할:** 로봇의 운동학 및 경로 계획을 담당하는 고수준 제어 계층

Movel2는 ROS2 위에서 동작하며, 로봇의 URDF(xacro, SRDF)모델을 기반으로  
Forward /Inverse Kinematics, 경로 계획(Motion Planning), 충돌 감지, Trajectory Execution  
→ Gazebo 시뮬레이션과 연동 시 실제 물리 환경에서 계획된 궤적을 테스트할 수 있음

## ROS2

(노드 관리, 통신 통합 프레임워크)

**역할:** 로봇 소프트웨어의 통신과 모듈 관리를 담당

노드간 Pub/Sub 메시지 통신, 서비스/액션 호출관리, TF(Transform)및 센서 데이터 브로커 역할  
ROS2는 센서, 제어기, 인식, 플래닝 등 각 기능 모듈이 독립적으로 개발되고 상호 작용  
운영체제 위의 미들웨어 → DDS(Data Distribution Service) 기반 통신을 사용

## 운영환경계층

(Ubuntu 22.04 환경)

**역할:** 전체 시스템의 기반이 되는 OS환경

ROS2 Humble 버전은 Ubuntu 22.04 LTS 환경을 공식 지원  
드라이버, 네트워크, 그래픽, CUDA 등 하위 리소스 접근을 모두 OS가 관리  
예) 건물의 '기초 토대'에 해당, 상위계층은 이 위에서만 작동 가능

# 개발환경 구성

## ◆ Debian 기반 리눅스 배포판

### ◆ 특징 및 장점

- 사용자 친화적: 기본적인 소프트웨어가 포함되어 있어 쉽게 진입 가능
- 무료: 오픈소스 커뮤니티에 기반하여 무료로 사용가능
- 보안: 리눅스 배포판 특성상, 안정적이고 보안적인 장점
- 다양한 플랫폼 지원: 서버에서 IoT에 이르는 다양한 플랫폼 지원

### ◆ 버전과 코드명

- `lsb_release -a` 명령어로 자신의 버전과 코드명 알기
- 현존하는 안정적이고 지원기간이 긴 버전을 택함
  - ✓ Ubuntu-22.04.x 버전: 안정적이고 지원기간이 가장 길다
  - ✓ Ubuntu-20.04.x 버전: 안정적이고 많은 예제 보유

#### Current

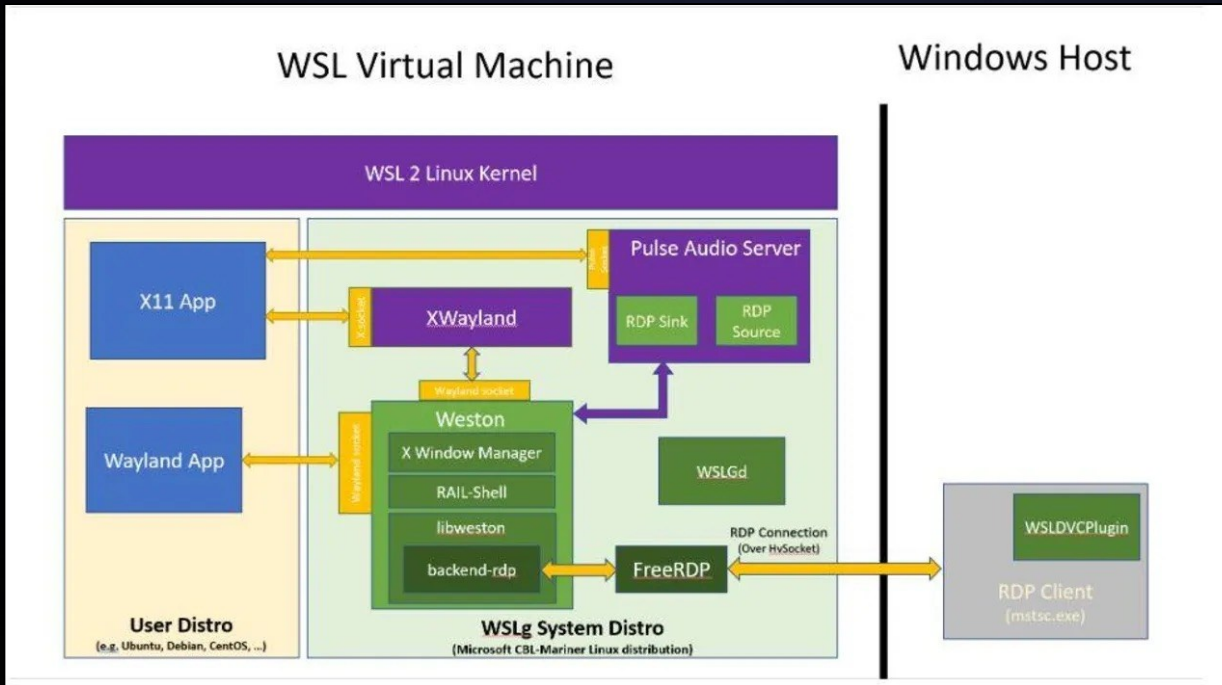
| Version                   | Code name       | Docs                          | Release            | End of Standard Support | End of Life |
|---------------------------|-----------------|-------------------------------|--------------------|-------------------------|-------------|
| Ubuntu <b>24.10</b>       | Oracular Oriole | <a href="#">Release Notes</a> | October 10, 2024   | July 2025               | July 2025   |
| Ubuntu <b>24.04.1 LTS</b> | Noble Numbat    | <a href="#">Release Notes</a> | August 29, 2024    | June 2029               | April 2036  |
| Ubuntu 24.04 LTS          | Noble Numbat    | <a href="#">Release Notes</a> | April 25, 2024     | June 2029               | April 2036  |
| Ubuntu <b>22.04.5 LTS</b> | Jammy Jellyfish | <a href="#">Release Notes</a> | September 12, 2024 | June 2027               | April 2034  |
| Ubuntu 22.04.4 LTS        | Jammy Jellyfish | <a href="#">Release Notes</a> | February 22, 2024  | June 2027               | April 2034  |
| Ubuntu 22.04.3 LTS        | Jammy Jellyfish | <a href="#">Release Notes</a> | August 10, 2023    | June 2027               | April 2034  |
| Ubuntu 22.04.2 LTS        | Jammy Jellyfish | <a href="#">Release Notes</a> | February 23, 2023  | June 2027               | April 2034  |
| Ubuntu 22.04.1 LTS        | Jammy Jellyfish | <a href="#">Release Notes</a> | August 11, 2022    | June 2027               | April 2034  |
| Ubuntu 22.04 LTS          | Jammy Jellyfish | <a href="#">Release Notes</a> | April 21, 2022     | June 2027               | April 2034  |
| Ubuntu <b>20.04.6 LTS</b> | Focal Fossa     | <a href="#">Changes</a>       | March 23, 2023     | April 2025              | April 2032  |
| Ubuntu 20.04.5 LTS        | Focal Fossa     | <a href="#">Changes</a>       | September 1, 2022  | April 2025              | April 2032  |
| Ubuntu 20.04.4 LTS        | Focal Fossa     | <a href="#">Changes</a>       | February 24, 2022  | April 2025              | April 2032  |
| Ubuntu 20.04.3 LTS        | Focal Fossa     | <a href="#">Changes</a>       | August 26, 2021    | April 2025              | April 2032  |
| Ubuntu 20.04.2 LTS        | Focal Fossa     | <a href="#">Changes</a>       | February 4, 2021   | April 2025              | April 2032  |
| Ubuntu 20.04.1 LTS        | Focal Fossa     | <a href="#">Changes</a>       | August 6, 2020     | April 2025              | April 2032  |
| Ubuntu 20.04 LTS          | Focal Fossa     | <a href="#">Release Notes</a> | April 23, 2020     | April 2025              | April 2032  |
| Ubuntu <b>18.04.6 LTS</b> | Bionic Beaver   | <a href="#">Changes</a>       | September 17, 2021 | June 2023               | April 2030  |

# 개발환경 구성

## ◆ WSL 2 소개

- Windows Subsystem for Linux 2 (WSL 2)는 Windows 환경에서 리눅스 바이너리를 네이티브로 실행할 수 있는 가상화 기술
- WSL 2는 이전 버전인 WSL 1과 달리 전체 리눅스 커널을 포함하여 향상된 성능과 애플리케이션 호환성을 제공

## ◆ 구조



## ➤ 리눅스 커널:

- ✓ WSL 2는 실제 리눅스 커널을 사용
- ✓ Microsoft가 유지 관리
- ✓ Windows 업데이트를 통해 자동으로 업데이트

## ➤ 하이퍼바이저 플랫폼

- ✓ WSL 2는 Windows의 하이퍼바이저 기술을 이용  
→ 가벼운 가상 머신을 구동
- ✓ 전체 시스템 리소스를 차지하지 않고도 뛰어난 성능을 제공

## ➤ 호스트 시스템과의 통합

- ✓ 파일 시스템, 네트워킹, 입력/출력 등 여러 방면에서  
→ Windows 호스트 시스템과 긴밀히 통합

# 개발환경 구성

## ◆GUI Application 지원

### ➤ 필수조건 구성

- ✓ **Windows 10 Build 19044+** 또는 **Windows 11**이 필요
- ✓ **vGPU용 설치된 드라이버:**
  - Linux GUI 앱을 실행하려면 먼저 아래 시스템과 일치하는 드라이버를 설치
  - vGPU(가상 GPU)를 사용할 수 있으므로 하드웨어 가속 OpenGL 렌더링의 이점([사이트 확인](#))

### ➤ 새로 설치

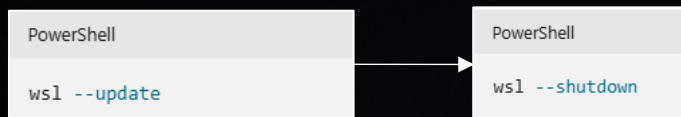
✓

```
PowerShell
ws1 --install
```

- ✓ 컴퓨터 재부팅 → id/pw 입력 (출처: <https://learn.microsoft.com/ko-kr/windows/wsl/install>)

### ➤ 기존 설치

✓





# 개발환경 구성

## ◆Linux GUI 앱 설치 및 실행

- 배포에서 패키지 업데이트

```
Bash
sudo apt update
```

- GIMP 설치: 이미지 편집

```
Bash
sudo apt install gimp -y
```

✓ 실행: gimp

- Gnome 텍스트 편집기 설치

```
Bash
sudo apt install gnome-text-editor -y
```

✓ 실행: gnome-text-editor 파일명

- X11 앱 설치: Linux 창 시스템

```
Bash
sudo apt install x11-apps -y
```

✓ 실행: xcalc, xclock, xeyes

- Nautilus: gnome 데스크탑 파일관리자

```
Bash
sudo apt install nautilus -y
```

✓ 실행: nautilus

- VLC 설치: 멀티미디어 파일 재생

```
Bash
sudo apt install vlc -y
```

✓ 실행: vlc



# 개발환경 구성

## ◆ 하드디스크 용량

- Ubuntu 22.04 LTS
    - ✓ 기본 설치 시 약 5GB에서 8GB 정도의 용량이 필요
    - ✓ 개발 도구와 업데이트, 추가 패키지를 포함할 경우 20GB 이상을 확보 필요
  - ROS2 Humble 기본 설치
    - ✓ 약 2GB에서 3GB의 공간이 필요
    - ✓ 관련된 다양한 ROS2 패키지 및 툴을 추가로 설치하면 최대 5GB까지 필요
  - Nav2 패키지 설치
    - ✓ Nav2의 기본 설치는 추가적으로 500MB~1GB 정도의 용량을 차지
    - ✓ Nav2 관련 시뮬레이션, 맵핑 툴, 경로 계획 및 제어 패키지 등의 의존성을 설치하면 약 1GB 이상이 추가
- 최소 20GB 이상, 안정적인 성능을 위해서는 30GB

## ◆ 메모리(RAM)용량

- Ubuntu 및 ROS2 구동 → Ubuntu와 ROS2 자체는 약 2GB의 메모리를 사용
  - Nav2 및 시뮬레이션 구동 → Nav2와 시뮬레이션 툴(RViz, Gazebo 등)을 동시에 구동하면 최소 4GB 이상의 메모리가 필요
  - 권장 메모리: 원활한 실행을 위해 8GB 이상의 RAM을 사용하는 것이 좋으며, 대규모 시뮬레이션을 위해서는 16GB 이상을 권장
- 최소 8GB 이상, 안정적인 성능을 위해서는 16GB

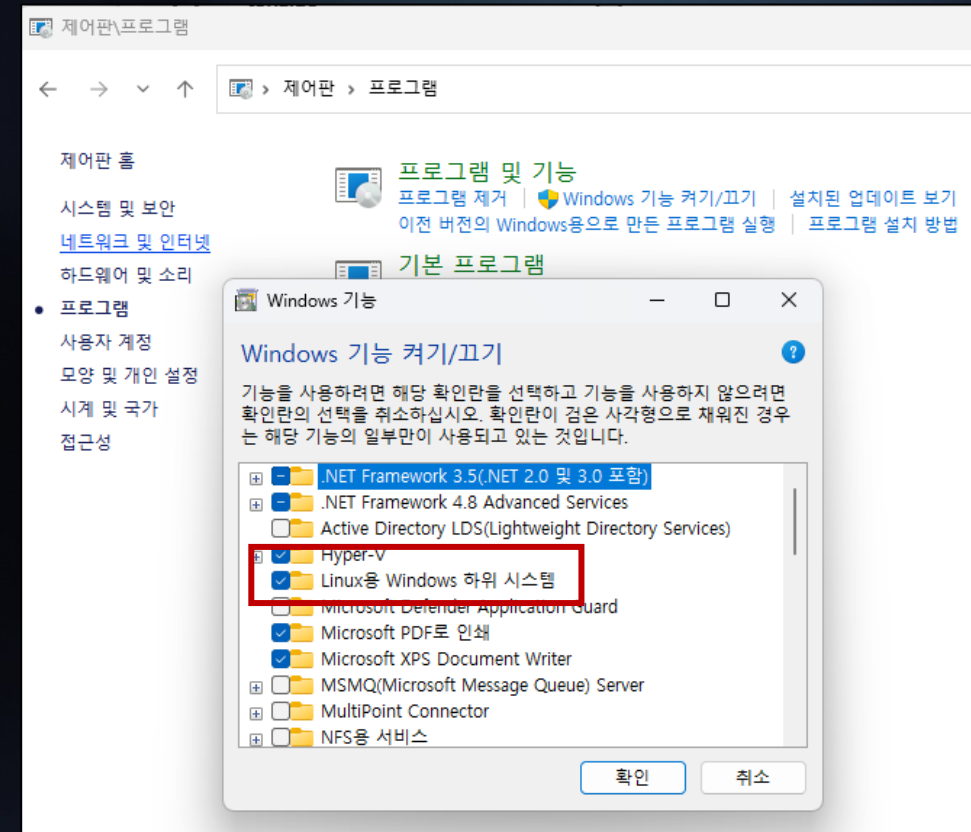
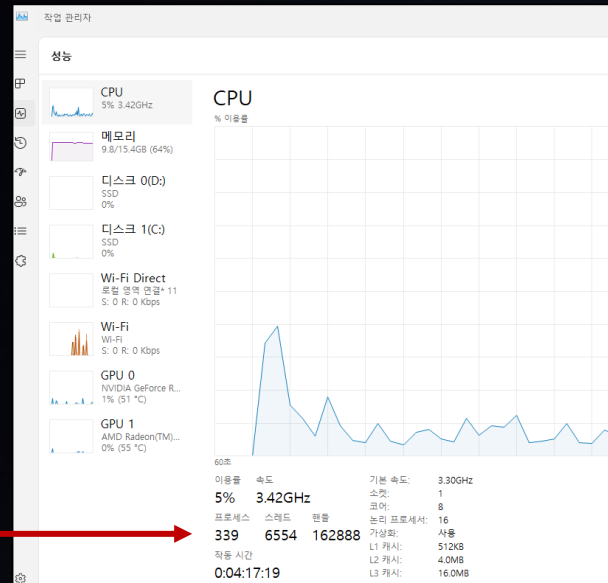
# 개발환경 구성

## ◆ Windows + Ubuntu 22.04T 설치

- WSL(Window Service for Linux): 윈도우에서 리눅스 가상머신을 돌릴 수 있도록 공식지원
- 지원 조건
  - ✓ Windows 10 1903 이상, 빌드 18362 이상 또는 Windows 11  
(Windows 10 Home, Windows 11 Home 버전도 사용가능)
  - ✓ 2. Hyper-V 가상화(Hyper-V Virtualization support)를 지원하는 컴퓨터

60초

|            |         |          |         |
|------------|---------|----------|---------|
| 이용률        | 속도      | 기본 속도:   | 3.30GHz |
| 7%         | 3.70GHz | 소켓:      | 1       |
| 프로세스       | 스레드     | 코어:      | 8       |
| 338        | 6588    | 논리 프로세서: | 16      |
| 작동 시간      |         | 가상화:     | 사용      |
| 0:04:18:29 |         | L1 캐시:   | 512KB   |
|            |         | L2 캐시:   | 4.0MB   |
|            |         | L3 캐시:   | 16.0MB  |



# 개발환경 구성

## ◆ WSL(Window Service for Linux) 설치하기

[단계1] Powershell을 관리자 권한으로 열어준다.

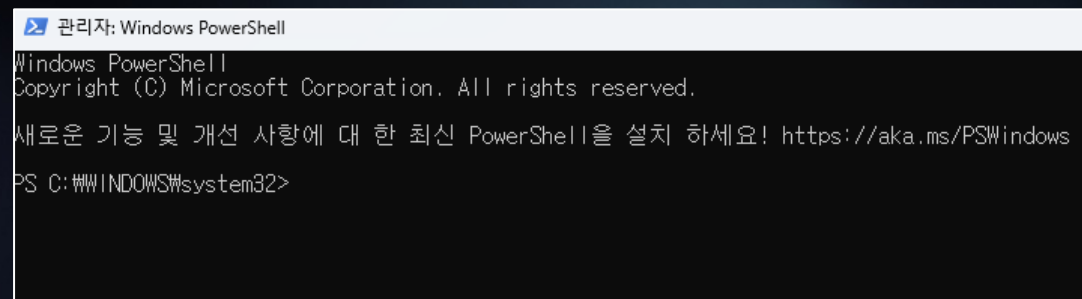
[단계2] 명령어: `wsl - - install` 실행

[단계3] 설치가 완료되었다고 메시지가 출력되면, 시스템 재부팅

(현재 배포중인 리눅스 배포판 확인) 명령어 `wsl -list --online`

[단계4] 설치를 위해 다음 명령어 실행: `wsl -install -d Ubuntu` : 재부팅 필요함

[단계5] 재부팅 후 검색 창에서 설치된 Ubuntu를 검색하여 실행하면 초기 계정 및 비밀번호 설정



```
관리자: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows
PS C:\WINDOWS\system32>
```

# 개발환경 구성

## ◆WSL 저장 경로 변경(C:\w → D:\w)

[단계1] WSL Ubuntu 확인하기: ] Powershell을 관리자 권한으로 열어준다. → 명령어: wsl -i -v

[단계2] 명령어: wsl - - shutdown 실행

[단계3] D:\w드라이브로 이동 후 다음 명령어 실행

- git clone https://github.com/pxlrbt/move-wsl.git
- cd move-wsl
- ./move-wsl.ps1
- 위의 명령어가 안되면 : Set-ExecutionPolicy Unrestricted 실행

[단계4] ./move-wsl.pl1 실행

[단계5] 옮기려고 한 내용을 선택(번호로 되어 있음. 번호 선택)후

→ D:/원하는 디렉토리명/ [Enter] y 입력

→ 자동으로 옮기는 것이 실행됨

[단계6] 입력한 디렉토리에 가서 확인

[단계7] 재접속 → root계정으로 접속됨.

→ ubuntu2204 config --default-user 입력했던 계정

# 개발환경 구성

## ◆ Docker 사용?

Docker에서 ROS2 Humble을 사용하는 이유

- 격리된 환경
- 간편한 설치 및 삭제
- 일관성 유지
- WSL과의 호환성: Ubuntu의 WSL 환경에서 Docker 컨테이너를 사용하면

Gazebo와 같은 시뮬레이션 프로그램을 사용할 때도 환경 설정이 수월

### ➤ Docker를 사용하지 않을 때의 장점

- 성능문제
- GUI 및 하드웨어 접근: ROS2와 연동된 Gazebo 시뮬레이션이나 RViz와 같은 시각화 도구는 GPU 자원이 필요  
Docker 환경에서는 GPU 접근이 복잡하거나 제한적
- 로컬 네트워크 구성의 복잡성

→ 여러 개의 독립된 프로젝트를 진행하거나 협업 프로젝트 진행으로 ROS2환경이 일관되어야 할 때 사용



# 개발환경 구성

## ◆ locale 설정

[단계1] `sudo apt update && sudo apt install locales`

[단계2] `sudo locale-gen en_US en_US.UTF-8`

[단계3] `sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8`

[단계4] `export LANG=en_US.UTF-8`

[단계5] locale 설정이 UTF-8로 잘 되어있는지 확인 : `locale`

## ◆ Source설치 : Ubuntu Universe Repository 활성화

[단계1] `sudo apt install software-properties-common`

[단계2] `sudo add-apt-repository universe`

## ◆ ROS2 GPG키 추가

[단계1] `sudo apt update && sudo apt install curl -y`

[단계2] `sudo curl -sSL <https://raw.githubusercontent.com/ros/rosdistro/master/ros.key> -o /usr/share/keyrings/ros-archive-keyring.gpg`

## ◆ repository를 추가

[단계3] `echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu`

`+ $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null`



# 개발환경 구성

## ◆Humble 설치 (Ubuntu 22.04환경 터미널에서 실행)

[단계1] Ubuntu 업데이트: 시스템 업데이트.

→ `sudo apt update && sudo apt upgrade -y`

[단계2] Desktop 버전을 설치

→ `sudo apt install ros-humble-desktop`

[단계3] Development tool을 설치

→ `sudo apt install ros-dev-tools`

[단계4] 환경설정: 최종적으로 setup script를 sourcing

→ `source /opt/ros/humble/setup.bash`

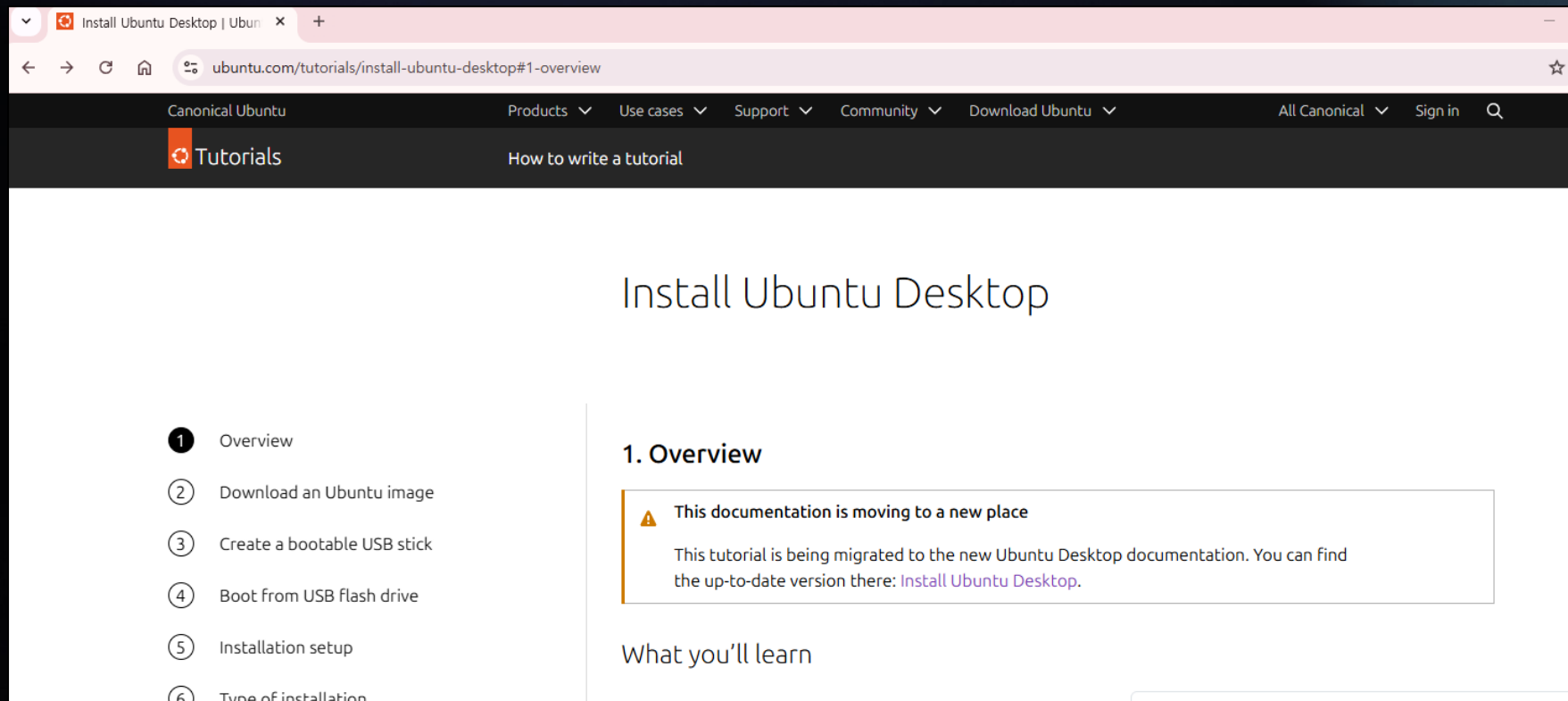
[단계5]매 터미널에 실행하기가 번거롭다 : bashrc에 추가

→ `echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc`

# 개발환경 구성

## ◆ Ubuntu 22.04 Desktop 버전 설치

- 아래의 링크를 참고하여 우분투 설치 USB를 만들고 설치합니다.
- <https://ubuntu.com/tutorials/install-ubuntu-desktop> 인스톨 가이드를 참고해서 설치합니다.

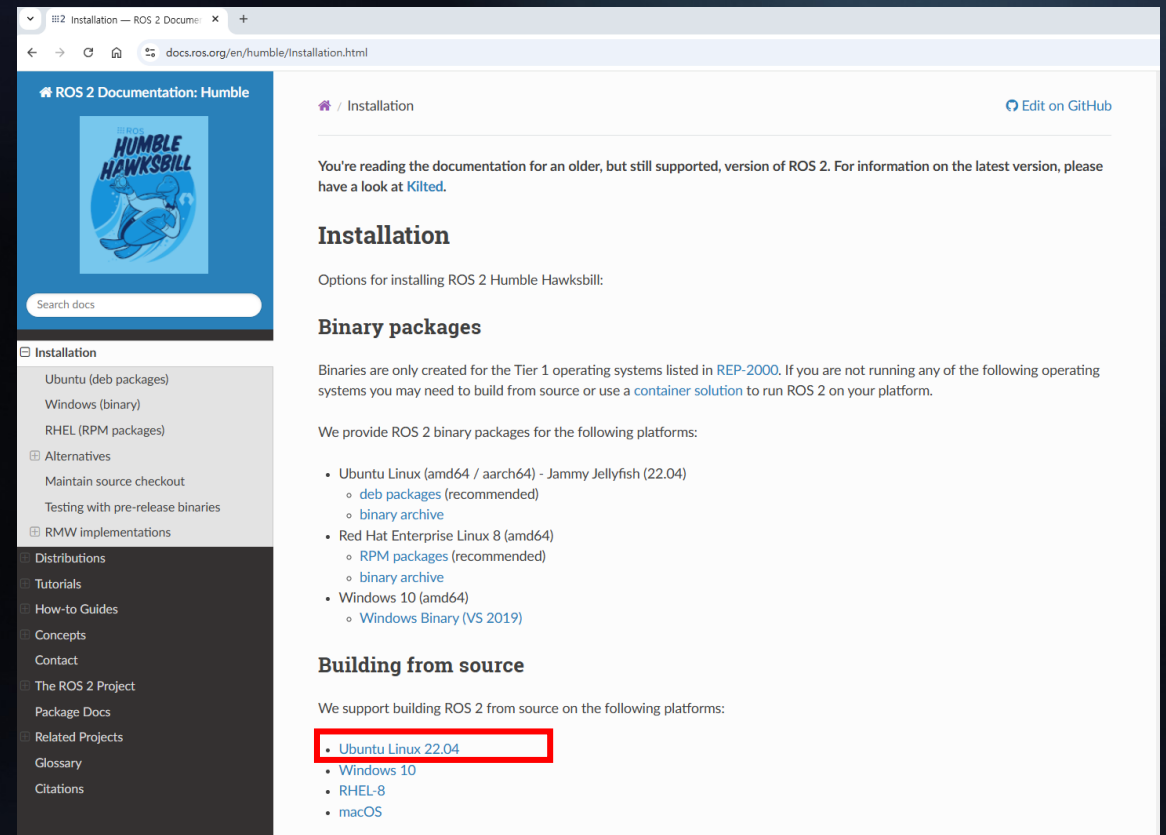


# 개발환경 구성

## ◆ ROS2 Humble 버전 다운로드 사이트

- <https://docs.ros.org/en/humble/Installation.html> →
- Ubuntu Linux 22.04 버전을 다운로드 합니다.
- <https://docs.ros.org/en/humble/Installation/Alternatives/Ubuntu-Development-Setup.html>

→ 여기 부분을 따라서 진행하시면 됩니다.



# ROS2

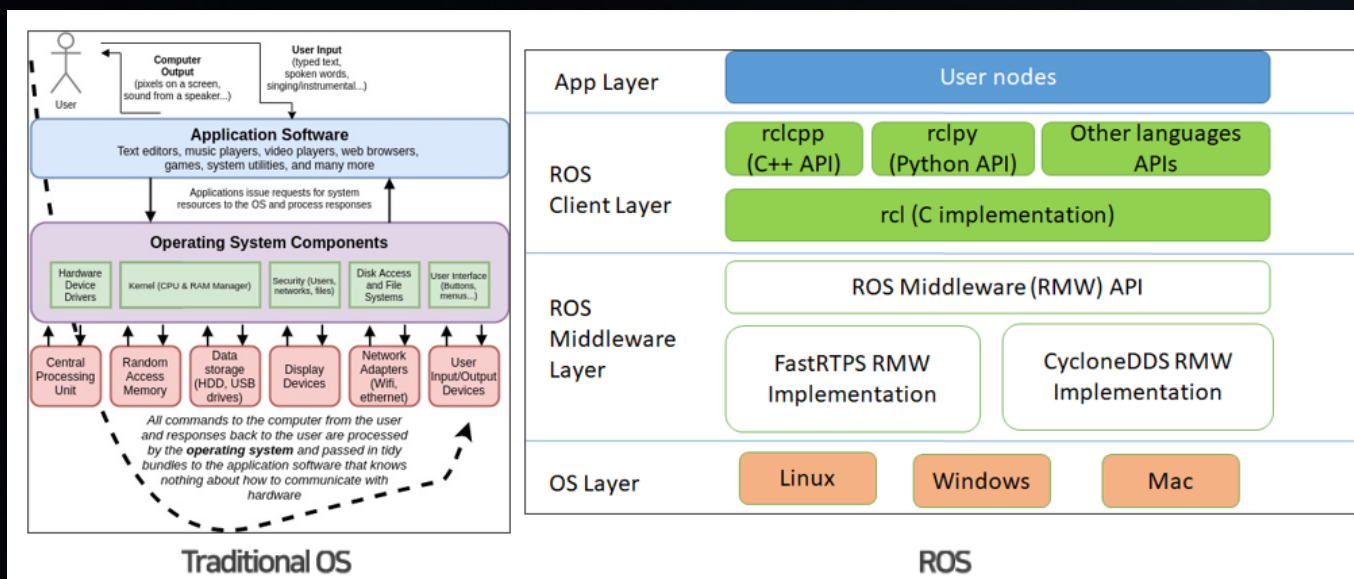
# (Robot Operating System)



# ROS2 소개

## ◆ ROS2 개요

- ROS = Robot Operating System이며, 로봇에서 동작하는 OS
- OS는 Windows, Linux와 같이 CPU등의 하드웨어와 사용자사이의 APP이나 S/W등을 관리
- ROS는 전형적인 OS위에서 동작하는 일종의 S/W이기 때문에 일반적인 OS와는 차이가 있음
- 로봇 하드웨어(Sensor, Monitor등)를 추상화 하고, 여러 S/W를 하드웨어와 연결하고 관리하는 측면에서는 OS와 동일
- ROS는 Robot S/W를 관리하고 하드웨어와 쉽게 상호 작용을 할 수 있도록 도와주는 역할을 수행.





# ROS2

## ◆ ROS2 소개

- 로봇 운영체제는 Robot Application을 구축하기 위한 S/W 라이브러리와 도구 세트
- ROS 패키지에 대한 참조(<https://index.ros.org/packages/#humble>)
- 플랫폼 별 Binary Package 제공

✓ Ubuntu Linux – 코드명(Jammy Jellyfish (22.04))

→ deb 패키지 (권장) / [binary archive](#)

### ➤ 버전 및 코드명제

✓ EOL(End of Life) date를 참고하여 버전을 결정

\* 출처: <https://docs.ros.org/en/humble/Releases.html>

| Distro              | Release date   | Logo                                                                                  | EOL date           |
|---------------------|----------------|---------------------------------------------------------------------------------------|--------------------|
| Jazzy Jalisco       | May 23rd, 2024 |    | May 2029           |
| Iron Irwini         | May 23rd, 2023 |    | November 2024      |
| Humble Hawksbill    | May 23rd, 2022 |   | May 2027           |
| Galactic Geochelone | May 23rd, 2021 |  | December 9th, 2022 |
| Foxy Fitzroy        | June 5th, 2020 |  | June 20th, 2023    |



# ROS2



## ROS2 Humble버전 선택

### ➤ 지원되는 Humble버전

- ✓ 주로 Ubuntu 22.04 LTS와 호환되며, 이 버전은 안정성과 장기 지원이 중요한 프로젝트에 적합
- ✓ 안정성에 중점을 둔 기능이 포함되어 있습니다. 다양한 DDS(Data Distribution Service) 구현과의 호환성을 지원
- ✓ 안정적인 내비게이션 및 시뮬레이션 패키지를 제공

### ➤ 기능적 업데이트

- ✓ 안정성에 중점을 둔 기능이 포함
- ✓ 다양한 DDS(Data Distribution Service) 구현과의 호환성을 지원
- ✓ 안정적인 내비게이션 및 시뮬레이션 패키지 제공

### ➤ 패키지 호환성과 지원

- ✓ 장기적으로 안정성을 유지하기 위한 다양한 패키지가 충분히 테스트되어 있음
- ✓ 자율주행이나 산업용 로봇 프로젝트에서 더 안정적

→ROS2 Humble에서 Gazebo Classic과 Ignition Gazebo 모두 사용할 수 있으며 연동도 원활

→특히, WSL환경에서 ROS2와 Gazebo를 통합해 사용 환경도 지원 원활

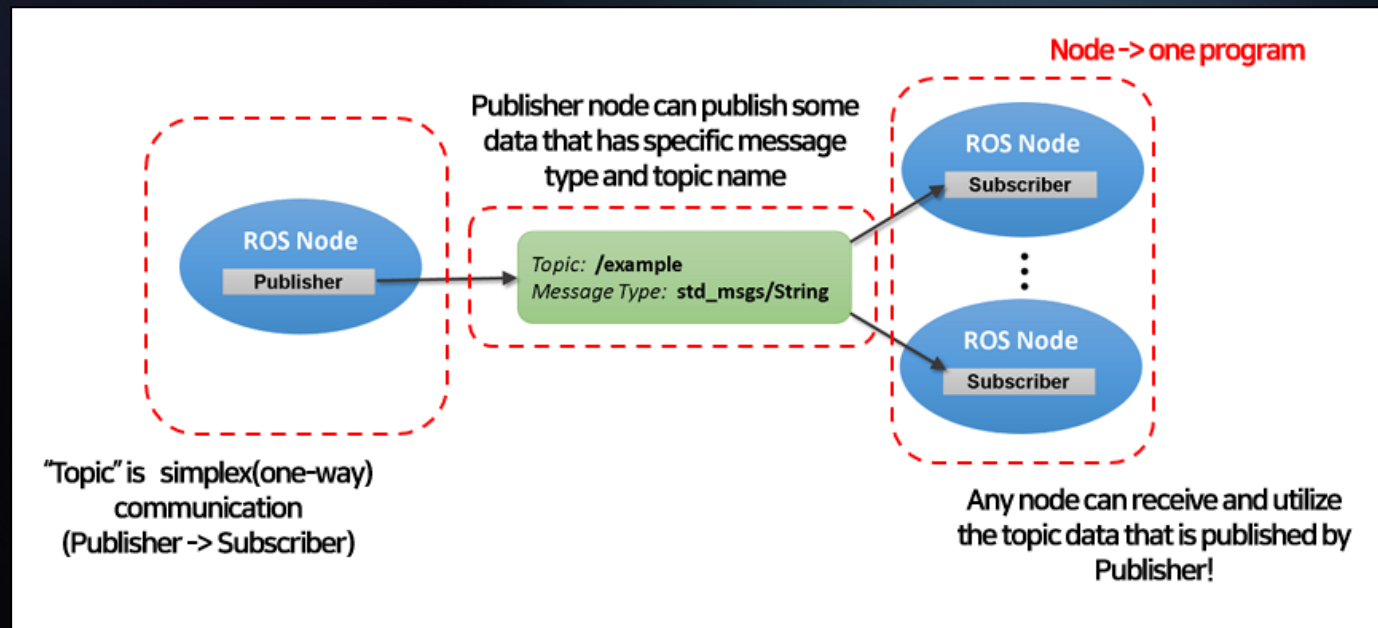
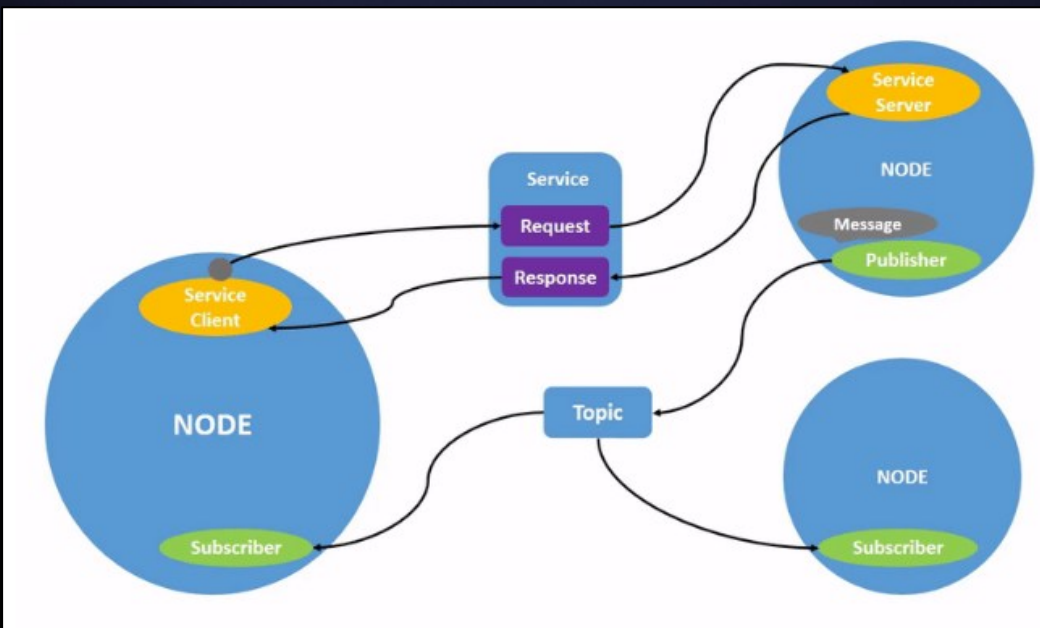
# ROS2 동작방식

## ◆ 개념 1: 노드와 패키지

- ROS에서 노드(node)는 어떤 프로그램의 최소 단위 → 노드들이 모여서 큰 기능을 하는 패키지
- 예) 로봇이 검정색 공을 찾아 들어올리도록 하는 패키지(패키지 내의 노드들..)
  - ✓ 카메라 영상에서 검정색 공을 찾는 노드
  - ✓ 특정 위치의 물체로 로봇을 움직여 접근하게 하는 노드
  - ✓ 특정 물체를 들어올리는 노드 등.

# ROS2 동작방식

## ◆ 개념 1: 노드와 패키지



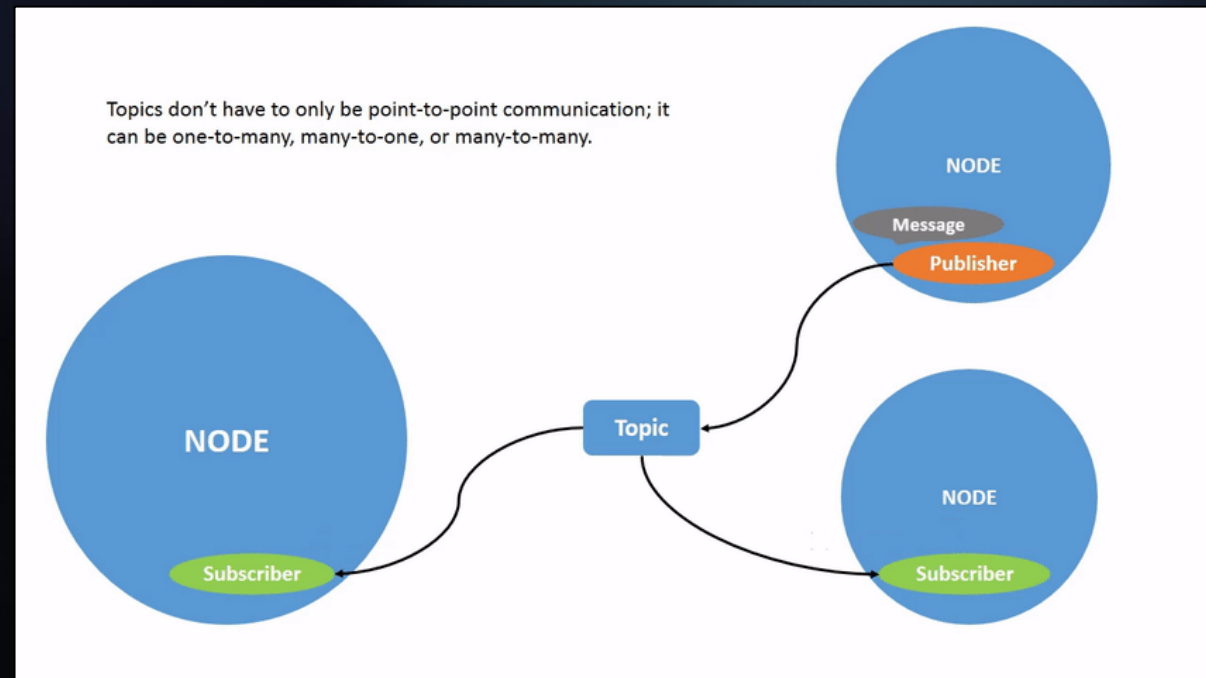
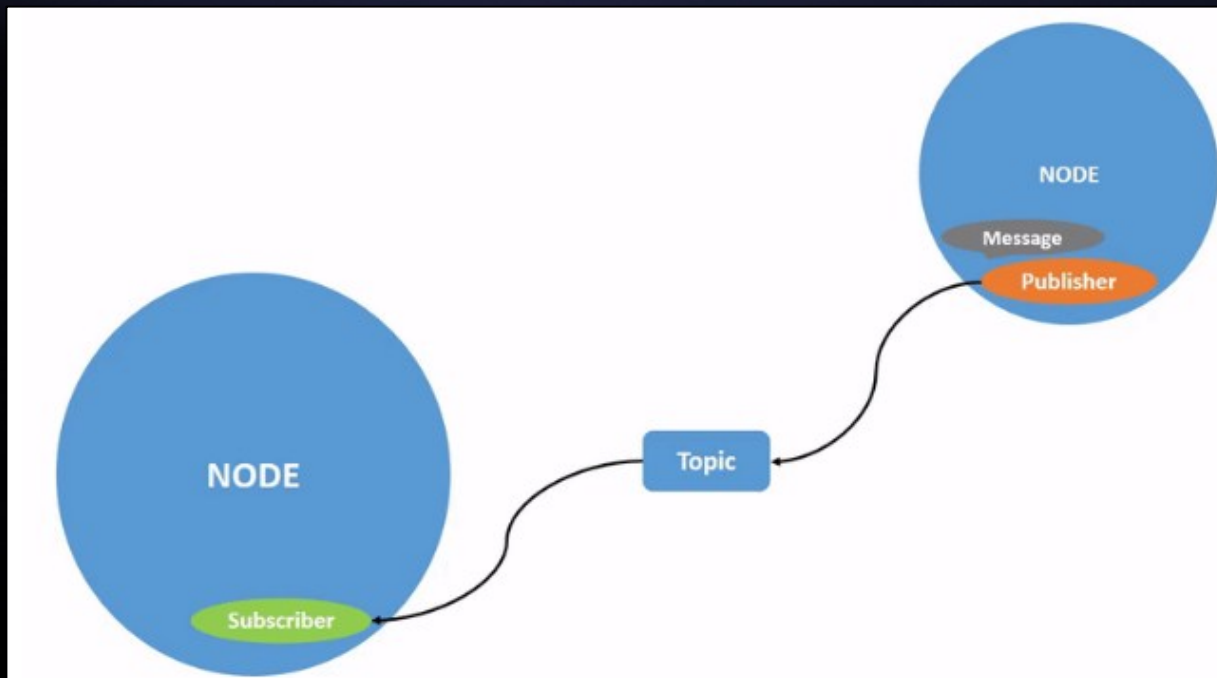
# ROS2 동작방식

## ◆ 개념2 : 토픽

- ROS의 가장 기본적인 통신 방법
- publisher와 subscriber로 구성
- publisher로부터 메시지(데이터)를 담은 토픽이 발행
  - subscriber가 받아 사용: 단방향 통신
- Publisher는 브로드캐스팅(계속 소리침) : subscriber 존재 유무 상관없이
- 이 소리는 아무나 들을 수 있듯이 → 아무나 subscriber가 되어 토픽을 받아 사용가능
- subscriber가 늘어나도 publisher의 소스코드를 변경할 필요가 없음 → 두 가지가 하나의 머신에 존재하는 것도 가능함

# ROS2 동작방식

## ◆ 개념2 : 토픽





# ROS2 동작방식

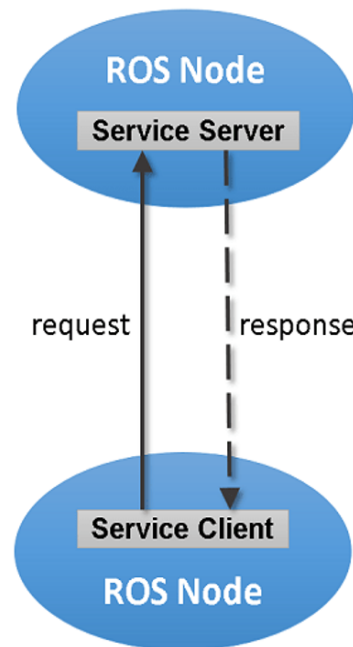
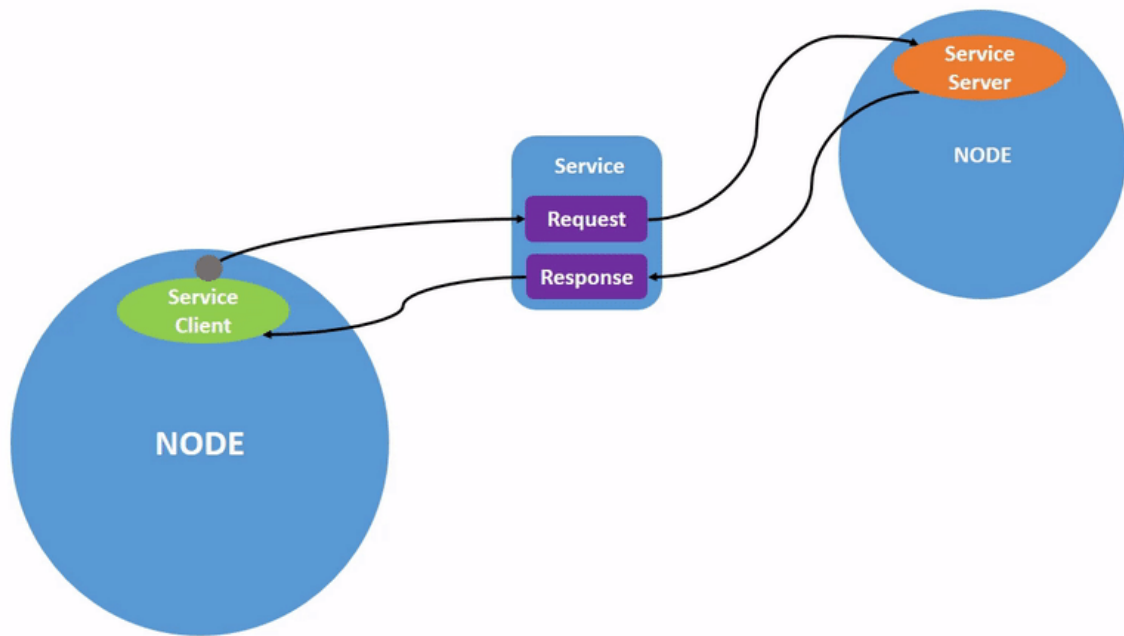
## ◆ 개념3 : 서비스

- 서비스에는 서버와 클라이언트가 존재
- 클라이언트: 서버에 어떠한 연산을 해달라고 요청을 보냄
- 서버: 클라이언트의 요청을 받아 어떤 연산을 수행하고
  - 그 결과를 돌려보냄
- 동작 순서 예시
  - ✓ 클라이언트가 서버에 요청을 보냄
  - ✓ 서버가 요청을 받으면 콜백함수가 동작하여 특정 작업을 수행함
  - ✓ 작업이 완료되면 서버에서 클라이언트로 그 결과가 송신됨
  - ✓ 클라이언트에서 작업 결과를 받아 활용



# ROS2 동작방식

## ◆ 개념3 : 서비스



Service Name: `/example_service`  
Service Type: `roscpp_tutorials/TwoInts`

Request Type: `roscpp_tutorials/TwoIntsRequest`  
Response Type: `roscpp_tutorials/TwoIntsResponse`

1. Client send the request to the server
2. Server does some task that client requests
3. When the task is completed, the server returns the result

# ROS2 동작방식

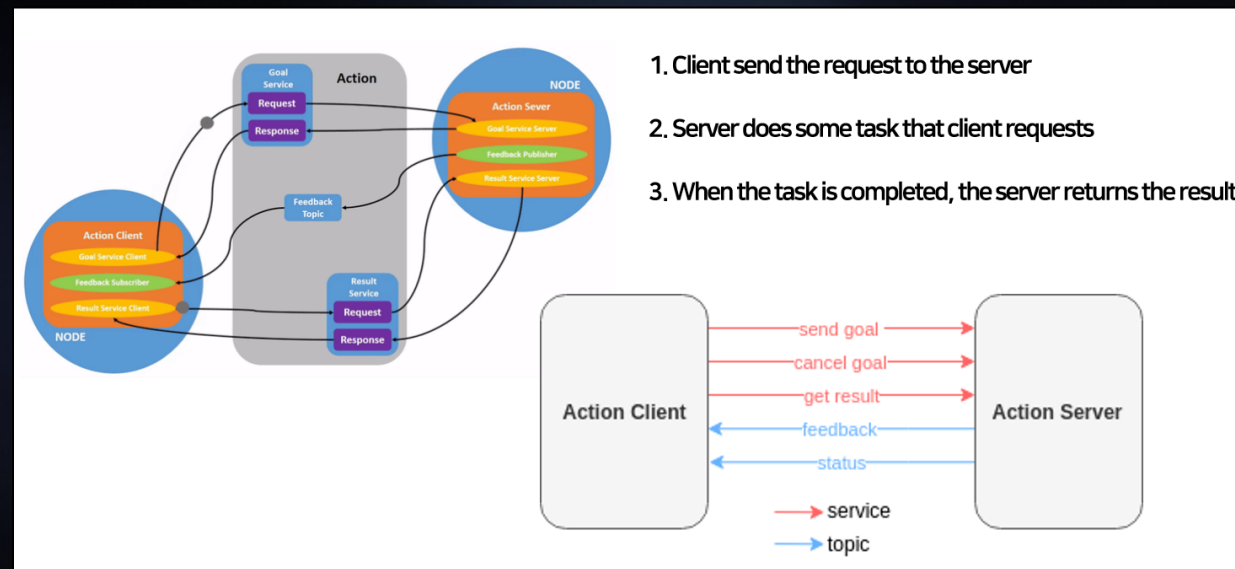
## ◆ 개념4 : 액션

- 액션에는 서버와 클라이언트가 존재 (서비스와 비슷)
- 클라이언트: 서버에 어떠한 연산을 해달라고 요청을 보냄
- 서버: 클라이언트의 요청을 받아 어떤 연산을 수행하고

→ 그 결과를 돌려보냄

### ➤ 동작 순서 예시

- ✓ 클라이언트가 서버에 요청을 보냄
  - ✓ 서버가 요청을 받으면 콜백함수가 동작하여 특정 작업을 수행함
  - ✓ 작업 중간마다 그 상태를 서버에서 클라이언트로 피드백을 보냄
  - ✓ 작업이 완료되면 서버에서 클라이언트로 그 결과가 송신됨
  - ✓ 클라이언트에서 작업 결과를 받아 활용
- 액션에서는 서비스와 달리 서버가 작업하는 중간에 그 상태를 피드백으로 클라이언트에게 알려 줄 수 있음
  - 중간에 작업의 취소도 가능함
  - 따라서 시간이 오래 걸리는 작업을 할 때 알맞음



# ROS2 동작방식

## ◆ 파라미터

- node의 구성하는 값
- 노드에서 사용하는 파라미터를 의미
- 매개변수를 노드 설정이라고 판단
- 노드는 매개변수를 정수, 부동 소수점, Boolean, 문자열 등으로 저장 가능
- 파라미터 서버에서 관리하고 프로그램이 동작하는 도중에 실시간으로 변경 가능
- (ex. 모터가 돌아가는 최대 최소 속도의 값 파라미터)요청하는 것

(출처: ros2 tutorial, <https://docs.ros.org/en/humble/index.html>)